

CSCE 637 COMPLEXITY THEORY · MODULE 08 · LECTURE NOTES

Randomised Complexity

Does a coin help?

MODULE OUTCOMES

Define BPP, RP, co-RP, and ZPP and distinguish them.

Explain error amplification and why one-sided error matters.

Explain why BPP is believed to equal P.

Explain derandomisation and its connection to lower bounds.

State what randomness demonstrably buys.

1 The classes

Class	Error allowed	Canonical example
BPP	Two-sided error, bounded away from 1/2 by a constant.	Polynomial identity testing — the headline problem in BPP and not known to be in P.
RP	One-sided: it may say no when the answer is yes, and never says yes when the answer is no.	Compositeness — a found factor is a certainty, so a yes is never wrong.
co-RP	One-sided the other way: never wrong on a yes.	Primality by Miller–Rabin — a declared composite is certainly composite, and 'probably prime' is probabilistic.
ZPP	No error at all — but the <i>running time</i> is random.	$RP \cap co-RP$. Las Vegas algorithms: always correct, expected polynomial time.
PP	Two-sided error with <i>any</i> gap from 1/2, however small.	Too permissive to be useful — it contains NP . See the note.

PP is the instructive warning: 'a majority of random choices accept', with no requirement on the size of the majority, is a useless guarantee, because you cannot distinguish a probability of $1/2$ from $1/2 + 2^{-n}$ by sampling polynomially many times. So the 'bounded away' condition in BPP is doing essential work, and it is the part of the definition that looks arbitrary and is not.

ERROR AMPLIFICATION IS WHY THE CONSTANT DOES NOT MATTER

Run a BPP algorithm k independent times and take the majority. The error probability falls exponentially in k , by a Chernoff bound (CSCE 658's material).

So an error bound of $1/3$ and an error bound of 2^{-100} define exactly the same class — about a hundred repetitions converts one into the other, and a hundred is a constant factor. **Which makes BPP robust to the choice of constant**, in the same way Module 01 §1's robustness arguments work.

Which is precisely why the definition requires the error to be *bounded away from $1/2$* by a constant: if the gap is allowed to shrink with n , amplification requires polynomially many repetitions and the argument fails. **That is PP, and it is why PP is not a useful class.**

And the practical reading is that 2^{-100} is smaller than the probability of a cosmic-ray bit flip during the computation — so a randomised algorithm with amplified error is more reliable than the hardware running a deterministic one, which is the correct answer to the objection that randomised algorithms 'might be wrong'.

2 Where randomness demonstrably won

- **Primality testing.** Miller–Rabin was the practical algorithm for twenty-five years before AKS, and it is still what every cryptographic library uses, because the deterministic algorithm is far slower (Module 02 §2). **A case where the complexity result and the engineering choice diverge permanently.**
- **Polynomial identity testing.** Is this symbolic arithmetic expression identically zero? In BPP — evaluate at random points — and not known to be in P, which makes it the headline open case and the problem whose derandomisation would imply circuit lower bounds (§3).
- **Volume estimation of convex bodies.** Randomised approximation is possible, and deterministic approximation is provably not (within the oracle model) — a real and unconditional separation, which is rare and worth knowing about.
- **Quicksort's pivot choice, hashing, load balancing, fingerprinting, and skip lists** — where randomness buys simplicity and resistance to adversarial inputs rather than asymptotic power (Module 01 §3's quicksort note).
- **And cryptography, which needs randomness irreducibly** — a deterministic cipher with a fixed key is a lookup table, and key generation, nonces, and padding all require genuine entropy (CSCE 711). **Note the pattern across the list: randomness most often buys simplicity and adversary-resistance, and only rarely a complexity class** — which is exactly what §3 explains.

3 Why BPP is believed to equal P

BPP = P IS THE EXPECTATION, AND THE REASON IS PSEUDORANDOMNESS

A pseudorandom generator stretches a short random seed into a long string that no efficient algorithm can distinguish from a truly random one. Note that this is a *computational* notion of randomness, not a statistical one — the output is demonstrably not random and no efficient test can tell.

If such generators exist, a BPP algorithm can be run on pseudorandom bits derived from *every* seed in turn, and the majority taken — which is entirely deterministic, and polynomial-time because the seeds are short (logarithmically many bits) so there are polynomially many of them.

And strong enough generators follow from circuit lower bounds (Module 09): Nisan and Wigderson showed that if some problem in E requires circuits of size $2^{\epsilon n}$, then $BPP = P$.

So hardness implies derandomisation. The existence of hard problems is what makes randomness unnecessary — which is the most counterintuitive result in this course, and is genuinely beautiful: you would expect hardness and the usefulness of randomness to point the same way, and they point opposite ways.

THE INTUITION: a sequence is random ENOUGH if no efficient observer can tell it from random. That is a COMPUTATIONAL notion, not a statistical one -- and it is the whole idea.

SO: take a function f that is HARD to compute. Its output bits look unpredictable to anything that cannot compute it. Use f to stretch a short seed into a long string.

NISAN-WIGDERSON: if some f in E requires circuits of size $2^{\epsilon n}$, then there is a pseudorandom generator fooling all polynomial-size circuits -- which gives $BPP = P$.

THE SHAPE OF THE ARGUMENT

HARD PROBLEMS EXIST $\Rightarrow$ PSEUDORANDOMNESS EXISTS
 $\Rightarrow$ RANDOMNESS IS REMOVABLE

AND THE CONVERSE HOLDS PARTIALLY: derandomising polynomial identity testing would imply circuit lower bounds (Impagliazzo-Kabanets). So the two questions are tied together in both directions.

WHICH MEANS: whether randomness helps is not an independent question. It is the SAME question as whether hard problems exist -- and that is Module 09's subject.

4 What follows

- **Use randomised algorithms freely.** 2^{-100} error is below the hardware fault rate (§1's callout), and the randomised algorithm is usually simpler than the deterministic one — which is a real engineering benefit independent of any complexity question.
- **Prefer one-sided error when it is available.** A found factor is a certainty; a Miller–Rabin 'probably prime' is not — and the difference matters inside a protocol, where a false positive and a false negative have different consequences.
- **And distinguish Las Vegas from Monte Carlo.** Random runtime with a guaranteed-correct answer (ZPP) is usually preferable to guaranteed runtime with a possibly-wrong answer, because the first can be retried and the second cannot be detected.
- **Randomness does not escape NP-hardness.** BPP is not believed to contain NP, so a coin will not solve your NP-complete problem — which is the practical headline and the thing people most often hope for.
- **And the quality of your randomness matters.** A weak generator can void the guarantee entirely, which is a security problem rather than a complexity one (CSCE 711) and has caused real failures — so 'randomised algorithm' assumes a source you have actually checked.

THE BRIDGE TO CSCE 658

This module is the complexity-class view of randomness. CSCE 658 is the algorithm-design view — how to construct randomised algorithms, analyse them with concentration bounds, and bound their failure probabilities.

And the two meet at derandomisation: CSCE 658 covers the method of conditional expectations and limited independence, which remove randomness from *specific* algorithms rather than from a whole class — a concrete and achievable version of §3's conditional result.

So the question 'does randomness help?' has a class-level answer (probably not, conditionally on circuit lower bounds) and an algorithm-level answer (frequently, for simplicity and robustness).

Which are both correct and are about different things — one is about the existence of an equally fast deterministic algorithm and the other is about which algorithm you should write — and keeping them apart is the main thing to carry from this module into the next course.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Arora & Barak — chapters 7 and 20 (free draft)	The randomised classes, and the derandomisation chapter — the reference for §1 and §3.
Nisan & Wigderson — Hardness vs Randomness (free)	§3's central result, and the title is the whole point.

Resource	What it covers for this module
Impagliazzo & Kabanets — Derandomizing Polynomial Identity Tests Means Proving Circuit Lower Bounds (free)	§3's converse direction , which ties the two questions together.
Motwani & Raghavan — Randomized Algorithms	The algorithm-design companion , and CSCE 658's main text. Library copy.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement Miller–Rabin** and measure the error rate empirically against known composites.
- 2 **Amplify the error by repetition** and confirm the exponential decay.
- 3 **Compute how many repetitions give error below 10^{-20}** , and compare against a hardware fault rate.
- 4 **Implement randomised polynomial identity testing** and construct an expression where the symbolic approach is infeasible.
- 5 **Classify five randomised algorithms you know** as BPP, RP, co-RP, or ZPP.
- 6 **Explain why PP is not useful**, with the sampling argument.
- 7 **Derandomise a simple randomised algorithm** by trying all seeds of logarithmic length.
- 8 **Explain the hardness-to-randomness direction** in your own words.
- 9 **Find a case where a weak random source broke a system** and describe it.
- 10 **Write down whether you expect $BPP = P$** , with reasons.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define BPP, RP, co-RP, ZPP, and PP, and say which is useless and why.
- 2 Explain error amplification and why the constant does not matter.
- 3 Why must the error be bounded away from a half?
- 4 Give five places randomness demonstrably helped, and the pattern across them.
- 5 Why is BPP believed to equal P?
- 6 State Nisan–Wigderson and the shape of its argument.
- 7 Why is 'does randomness help' the same question as 'do hard problems exist'?
- 8 Give five practical readings.

- 9 Distinguish the class-level and algorithm-level answers.